Evaluación del módulo: *Fundamentos de Bases de datos relacionales*

Consigna del proyecto👨‍💻

alkemy

Proyecto: *Alke Wallet*

Módulo *: Fundamentos de bases de datos relacionales*

Situación inicial 📍

Como parte de un equipo de programadores recibiste el pedido de ***Alke Wallet*** de desarrollar ***el diseño de una base de datos relacional para gestionar un sistema de una Wallet*** la cual permitirá a sus usuarios almacenar ***y gestionar sus fondos, realizar transacciones y ver el historial de transacciones.*** Nuestro líder técnico ya cuenta con los requerimientos desagregados en un backlog de tareas listo para que comencemos la etapa de desarrollo.

Nuestro objetivo 📋

El objetivo de esta evaluación es diseñar el modelo conceptual, definir las relaciones entre las entidades de una wallet y crear la base de datos para almacenar la información que sea necesaria.

Requerimientos 🤝

La ***Alke Wallet*** deberá cumplir con una serie de características y requerimientos técnicos para garantizar la calidad y funcionalidad de la misma.

Primero, como requerimiento general tenemos que poder implementar todas las funcionalidades básicas que un usuario necesita para ***gestionar sus fondos, realizar transacciones y ver el historial de transacciones***, las cuales son:

→ Diseñar una Bases de Datos que garantice la coherencia y la integridad de los datos

→ Crear una conexión a una Bases de Datos llamada ***Alke Wallet***

→ Crear Entidades:

1. **Usuario**: Representa a cada usuario individual del sistema de monedero virtual.

- ***Atributos:***

  `user_id (clave primaria)`

  `nombre`

```
correo electrónico
contraseña
saldo.
```

2. **Transacción**: Representa cada transacción financiera realizada por los usuarios.

```
- Atributos:
transaction_id (Primary Key)
sender_user_id (Foreign Key referenciando a User)
receiver_user_id (Foreign Key referenciando a User)
importe
 transaction_date.
```

3. **Moneda:** Representa las diferentes monedas que se pueden utilizar en el monedero virtual.

```
- Atributos:
currency_id (Primary Key)
currency_name
currency_symbol
```

→ Crear consultas SQL para:

- Consulta para obtener el **nombre de la moneda elegida** por un usuario específico.
- Consulta para obtener **todas las transacciones registradas.**
- Consulta para obtener todas las **transacciones realizadas por un usuario específico**.
- Sentencia DML para **modificar el campo *correo electrónico*** de un usuario específico.

- Sentencia para **eliminar los datos de una transacción** (eliminado de la fila completa)

🚨**¡Importante!** Todas las sentencias SQL hechas tanto para crear las entidades (tablas) como para manipular los datos de la DB (crear, leer, modificar, eliminar, etc) deben quedar registradas en un documento en formato Word.

Después de diseñar el modelo conceptual y de definir las relaciones, podrás comenzar a implementar la base de datos de la Wallet utilizando SQL. Puedes crear las tablas necesarias, configurar las claves primarias y las claves externas, y añadir restricciones para mantener la integridad de los datos.

Paso a paso 👣

Este proyecto será avanzado paso a paso en cada clase y podrás completarlo progresivamente a través de los espacios ***asincrónicos*** en la herramienta de **Visual Studio Code**. También tendrás espacios de consulta en las clases sincrónicas para despejar tus dudas.

Esto fue pensado para que llegues a completar esta *Evaluación Integradora* de manera progresiva, resolviendo el desafío a lo largo del módulo llegando al final de este con gran parte del trabajo resuelto.

1. **Las bases de datos relacionales** (Lección 1)

   ↪ 📌 **Objetivo:** ejecutar sentencias en **SQLOnline** y organizar scripts `.sql` desde **Visual Studio Code**.

   ↪ 📍 **Tareas a desarrollar:**

   - Investigar qué es una BD relacional y sus ventajas frente a otros modelos.
   - Crear una cuenta (gratuita) en sqliteonline.com (modo MySQL 8) u otra plataforma equivalente.
   - Configurar Visual Studio Code con la extensión *SQLTools* (o similar) y crear el archivo `AlkeWallet.sql`.
   - Ejecutar en SQLOnline los comandos `CREATE DATABASE AlkeWallet; USE AlkeWallet;` y verificar la creación con `SHOW DATABASES;`.
   - Explorar objetos básicos mediante comandos `SHOW TABLES;` y `DESCRIBE <table>`.

- 💡 **Tarea Plus (opcional):** Realizar un esquema breve que relacione tablas, filas y columnas con ejemplos cotidianos.
- 📂 **Pantallas / Recursos iniciales**
  - Captura del primer `CREATE DATABASE` ejecutado exitosamente en SQLOnline.
  - Tabla comparativa: RDBMS libres vs. comerciales.

2. **Consultas a una o varias tablas** (Lección 2)
   - 📌 **Objetivo:** construir consultas SQL para recuperar información simple y compuesta (JOINs y sub-consultas) aplicando cláusulas `SELECT`, `WHERE`, `GROUP BY` y funciones de agregación.
   - 📍 **Tareas a desarrollar**
     - Ejecutar consultas `SELECT` básicas sobre la tabla `usuario`.
     - Implementar filtros dinámicos con `WHERE` y operadores lógicos.
     - Unir las tablas `transaccion` y `usuario` mediante `INNER JOIN`.
     - Practicar sub-consultas para obtener el total de transacciones por usuario.
   - **Tarea Plus** 💡 Crear una vista que muestre el top-5 de usuarios con mayor saldo.
   - 📂 **Pantallas / Recursos iniciales**
     - Captura del resultado de un `JOIN` exitoso.
     - Script de ejemplo con agregaciones (`COUNT`, `SUM`).

3. **Sentencias para la manipulación de datos y transaccionalidad** (Lección 3)
   - 📌 **Objetivo:** emplear DML (`INSERT`, `UPDATE`, `DELETE`) en transacciones controladas, aplicando los principios ACID para mantener la coherencia de la Wallet.
   - 📍 **Tareas a desarrollar**
     - Insertar registros de prueba en `usuario`, `moneda` y `transaccion`.

- Actualizar el saldo de un usuario luego de una transacción.
- Implementar una transacción con `START TRANSACTION`, `COMMIT` y `ROLLBACK`.
- Simular un error de integridad referencial y revertir la operación.

↪ **Tarea Plus** 💡 Diseñar un *script* que cargue 50 transacciones aleatorias usando secuencias.

↪ 📂 **Pantallas / Recursos iniciales**

- Captura de la consola mostrando un `COMMIT` exitoso.
- Tabla explicativa de las propiedades ACID.

4. **Sentencias para la definición de tablas (DDL) -** (Lección 4)

↪ 📌 **Objetivo:** diseñar y crear tablas `usuario`, `moneda` y `transaccion`, definiendo tipos de datos, claves primarias y foráneas, índices y restricciones de nulidad.

↪ 📍 **Tareas a desarrollar**

- Crear la base de datos `AlkeWallet` y seleccionar su uso.
- Definir la tabla `usuario` con `PRIMARY KEY (user_id)` y atributos básicos.
- Definir la tabla `moneda` (`currency_id`, `currency_name`, `currency_symbol`).
- Definir la tabla `transaccion` con claves foráneas a `usuario` y la moneda de la Wallet.
- Añadir restricciones `NOT NULL` e índices compuestos para optimizar búsquedas.

↪ **Tarea Plus** 💡 Modificar la tabla `usuario` para añadir la fecha de creación usando `ALTER TABLE`.

↪ 📂 **Pantallas / Recursos iniciales**

- Captura del resultado de `SHOW CREATE TABLE` o `DESCRIBE` de las tres tablas.
- Fragmento de código DDL completo con comentarios.

5. **El modelo entidad-relación** (Lección 5)

    - 📌 **Objetivo:** modelar conceptualmente el sistema Alke Wallet mediante un diagrama ER, que identifique entidades fuertes y débiles, establezca cardinalidades y transforme el modelo en un esquema relacional normalizado (3FN).

    - 📍 **Tareas a desarrollar**

        - Extraer requerimientos y listar entidades, atributos y relaciones.
        - Dibujar el diagrama ER con una herramienta libre (p. ej., *dbdiagram.io*, drawSQL) o con la extensión *draw.io* en VS Code.
        - Definir cardinalidades y opcionalidades entre `usuario`, `transaccion` y `moneda`.
        - Aplicar reglas de normalización hasta 3FN y documentar los cambios.
        - Generar el script DDL final de forma manual (o exportado desde la herramienta ERD) y probarlo en SQLOnline.

    - **Tarea Plus** 💡 Presentar el ERD exportado en PDF e incluirlo en el portafolio.

    - 📂 **Pantallas / Recursos iniciales**

        - Captura del diagrama ER completo.
        - Tabla de correspondencia ER → Relacional.

¿Qué vamos a validar?🔍

Con el objetivo de brindar transparencia sobre lo que se espera construir, este ítem responderá a la pregunta **¿Qué vamos a evaluar del producto? y ¿cuáles son los criterios de evaluación?**, por lo que será necesario detallar 3 aspectos claves:

1. **Aspectos técnicos:** Diseño de la base de datos, Integridad de los datos, utilización de identificadores de bases de datos, integridad referencial, utilización de SQL, DDL y DML para la manipulación de datos.

2. **Aspectos estructurales**: cumplimiento del principio ACID, es decir, que la base de datos debe ser atómica, consistente, aislada y durable. Es de suma importancia que el diseño de la base de datos y la relación entre sus tablas garantice la coherencia y la integridad de los datos.

Recursos 🎁

Tener en cuenta los tipos de relación:

TIPO	RELACIÓN	REPRESENTACIÓN
1:1	Una a una : La cardinalidad máxima en ambas direcciones es 1.	1 — ◇ — 1
1:N	Una a muchas: La cardinalidad máxima en una dirección es 1 y en la otra muchos.	1 — ◇ — N
N:M	Muchas a muchas: La cardinalidad máxima en ambas direcciones en muchos.	N — ◇ — M

Entregables ✅

Se debe entregar un documento en **formato Word** que contenga lo siguiente:

- **Consultas SQL:** Se incluirán todas las sentencias SQL utilizadas para crear y manipular la base de datos del Virtual Wallet.
- **Captura de pantalla:** Se adjuntará una foto (captura de pantalla) que muestre los primeros resultados obtenidos al ejecutar los queries.

El documento debe estar claramente organizado y etiquetado para evidenciar el proceso de implementación y los resultados obtenidos.

Portafolio 💼

¡Excelente! Puedes subir el entregable del proyecto "Alke Wallet" a tu portafolio. Asegúrate de presentar el proyecto "Alke Wallet" de manera clara, concisa y atractiva, destacando tus contribuciones y los aspectos más relevantes del proyecto.

¡Éxitos!

Nos vemos más adelante

alkemy